CI/CD / REPRODUCIBLE LABORATORY

CI/CD Manual

Git branching, pipeline controls and declared GitLab configuration.

Document control	Value
Document ID	MSC-CI-001
Document version	1.0.0
Released	2026-10-26
Source baseline	MSC-129; application and shared package versions remain 1.5.0
Owner	myscoutee-ci
Audience	Developers and operators reproducing the local GitLab laboratory
Execution scope	GitLab is the qualified scheduler. The newly supplied GitHub Actions conversion is inactive.
Credential policy	Public defaults and input mappings are versioned. Passwords, tokens, keys and target-specific local inputs are ignored by Git.

This manual describes the repository implementation. It is not a declaration that every optional test, target or external publication has been qualified.

Contents and ownership

Section	Page
1. Git branching and release integration	3
2. Pipeline jobs and manual controls	4
3. GitLab configuration and runners	5
4. Infrastructure and artifact flow	6
5. Daily operation, retention and evidence	7

Start with the right repository

The CI repository owns orchestration. Each product or dependency repository owns its build recipe, package metadata and config/ci declarations. The shared adapters call those existing owners; CI does not replace product packaging or introduce a second application implementation.

Owner	Files / responsibility
myscoutee-ci	ci/project.gitlab-ci.yml; ci/projects.json; scripts/project.py; provisioning, configuration and maintenance scripts
Each source repository	config/ci/project.json, pipeline.yml, variables.json and inactive github-actions.yml
myscoutee-microservices	Terraform/libvirt, k3s, domain-group image assembly, Helm and KVM deployment
myscoutee-packaging	DEB assembly, client tools, partial updates, signing and public verification
Backend / template	Independent composition, runtime, ports, credentials and deployment state

Related documents

The backend and template guides/publication/manuals directories contain the Laboratory Access Manual (MSC-LAB-001): URLs, distinct ports, login input locations and accepted target results. This CI manual explains the workflow behind those endpoints.

Detailed source references are under guides/development/ci/: configuration.md, dependencies.md, github-actions.md, kubernetes.md, runner-host.md, retention.md, verification.md and MSC-129-plan.md.

1. Git branching and releases

```
feature/task  -- push --> automatic candidate build
      |                     optional tests / manual VMware
      | merge request
      v
master        -- merge --> automatic candidate rebuild
      |
      | click release
      v
release/vX.Y.Z -- versioned candidate + tag + publication
      |
      +-- GitLab MR and GitHub PR --> master
                                      release branch removed
```

One integration branch

Use feature/* and merge requests into master; there is no develop branch. A feature build is an unpublished candidate. Its optional tests and manual VMware deployment allow review before integration. Feature pipelines cannot release, publish Pages/downstream dependencies, or deploy to Netcup, KVM or monolith.

The local GitLab feature/* rule grants protected-variable access only to trusted Maintainers (access level 40). This is a trusted laboratory policy, not permission to run arbitrary fork code with private deployment credentials.

Version planning is not publication

prepare plans the next version from stable source versions/tags. The commit-title markers release: major, release: minor and release: patch, or LAB_RELEASE_LEVEL, select the level; otherwise patch is used. From 1.5.0 these are 2.0.0, 1.6.0 and 1.5.1. Version/dependency edits stay in the isolated candidate until release integration.

One release operation

The manual release job publishes release/v<version>, its immutable tag and configured GitHub/GitLab assets and images, then integrates through a GitLab merge request and a GitHub pull request. It does not directly push master. Concurrent master changes are retained; a conflict leaves the release branch for resolution and retry.

Already-built release and merge commits use [skip ci] to avoid building another patch candidate. This is a commit-message directive, not an author name. The source version reaches master through integration. Successful integration removes the temporary release branches.

Release titles come from each project's release_name, for example MyScoutee Template 1.5.0. Optional release-note files do not gate a version-only release. Normal releases do not overwrite an existing release; the MSC-129 replacement of corrected 1.5.0 dependencies was an explicitly approved cleanup.

2. Jobs and manual controls

Job / stage	Behavior
prepare / dependencies	Automatic version and source/dependency plan; does not deploy.
build	Automatic normal project build plus lightweight artifact preparation. LAB_HEAVY_BUILD makes the exceptional heavy recipe manual (for example the admin DEB recipe).
test	Optional manual; allow_failure permits continuation. An unplayed test is not a pass.
package:deb / package:kvm	Optional manual assembly after build. Full installers and KVM domain images are explicit resource-intensive steps.
release / downstream	Manual release on protected master. Only its success triggers consumer pipelines; their normal builds are automatic, their releases remain manual.
infra:start / prepare:deploy	Optional manual asset preparation and VM reconciliation. It can run after dependencies without waiting for application build.
deploy:*	Manual target deployment. Requires the appropriate existing package/target; it does not implicitly build a DEB or provision missing infrastructure.
deploy:agent	Manual GitLab agent installation after KVM deployment.
verify:*	Explicit manual public verification after the corresponding deploy.
deploy:github	Manual frontend publication to permanent gh-pages/docs.

Stage order is not a mandatory infrastructure dependency

GitLab needs edges permit eligible jobs to run independently of visual stage order. Build does not depend on infra:start. Deployment checks the actual target, so an already-running KVM does not require another infra click. Missing prerequisites produce a failure with an actionable message.

Packaging is target-specific

A VMware partial update consumes existing named images. A full DEB requires package:deb first. KVM assembly retains independent domain-group images and compares real build inputs; unchanged groups are reused. The selected job does not mean every domain is rebuilt.

The release job is a blocking manual gate (allow_failure: false). Optional manual tests, packaging and deployment use allow_failure: true. A pipeline waiting for release can therefore show Blocked while its normal build has succeeded.

3. GitLab configuration

Declaration	Purpose
.env.example / compose.yaml	Pinned CE and runner images; HTTP 8929, SSH 2224, registry 5050; state and concurrency defaults.
ci/projects.json	22 project definitions, types, dependencies and GitHub destinations.
examples/variables.json	Public defaults, exact reviewed source pins and credential-file declarations.
Product config/ci/variables.json	Project and environment scopes, public port defaults, FILE variables and optional GitHub mappings.
ci/branches.json	Trusted feature-branch protection.
ci/kubernetes.json / .gitlab/agents/	Agent registration and Git-versioned Kubernetes access configuration.
Ignored .env / .local/private/	Local overrides and file locators; credential values never enter source.

```
bash scripts/lab.sh init
# Supply local paths in the ignored .env.
python3 scripts/setup.py

# Preview names/scopes, then reconcile configuration:
python3 scripts/configure.py
python3 scripts/configure.py --apply
```

The setup script creates/reconciles the myscoutee group, projects, source imports, runners and declared variables. Re-running uses the existing state. It does not deploy applications or publish a GitHub release. Later source imports/pushes can start pipelines.

Pinned shared library

Each thin pipeline entry includes the shared CI project at LAB_CI_REF. Jobs check out that reviewed commit. Committing a CI change is not enough to change the live group pin: configure.py --apply reconciles the declaration. The working tree must be reviewed and committed before setup.

Two runner registrations, four build slots

The Docker runner handles isolated validation. The native lab-build runner handles trusted builds with LAB_BUILD_CONCURRENCY=4. Four jobs can run within one registration. The dedicated host-account helper and ignored runner-host.env own host provisioning; application jobs do not wait for interactive sudo.

New commits can cancel interruptible work; project-build serializes builds within a project. Candidate resource groups serialize package/deploy/verify mutations on the same pipeline workspace. They do not remove all duplicate API-created pipelines or authorize simultaneous application targets on this host.

4. Infrastructure and artifacts

```
source --> isolated build --> local candidate files / Docker cache
                                |
                  manual deploy +--> selected target
                                |
                  manual release+--> GitLab packages / containers
                                +--> GitHub release / Packages / GHCR
                                +--> dependency consumers
```

Registry responsibilities

Storage	Contents and reuse
Candidate workspace / job artifacts	Unpublished files and local image cache. .lab/plan.json and artifact manifests describe the candidate; KVM assembly reports reuse/build decisions.
GitLab Package Registry	Versioned release packages and pinned infrastructure installer inputs. It is not a second candidate cache.
GitLab Container Registry / GHCR	Configured released Docker images, published by release. Unpublished images may remain in the native runner Docker daemon.
Terraform Module Registry	Small labkvm/libvirt module, automatically published by its owner. Module version is independent of application candidate version.
GitLab Terraform state	Private per-product deployment state. Backend and template share the module but use their own runtime configuration and project-scoped state.

Optional manual infrastructure

infra:start checks the pinned registry assets and reconciles Terraform. Missing installer inputs may be downloaded, verified and uploaded for later reuse. Existing registry entries avoid another upstream download. The operator import-infra-assets.py tool seeds the same registry from files already downloaded locally.

Deployment does not silently download multi-gigabyte installers. If the target or input is missing, the operator can run infra and retry. Reconciliation checks existing resources first; unchanged resources do not need replacement.

KVM and agent ownership

Domain packaging belongs to microservices and product composition. Required Mongo seed content belongs to application startup, separate from optional demo-data operations. deploy:agent connects GitLab to the existing cluster; the environment dashboard uses declared agent context and namespace. Stopping the VM naturally makes the agent unavailable.

5. Daily operation and evidence

Choose an operation

Goal	Operator sequence
Review a feature	Push feature/*; let build finish; optionally run test; manually deploy:vmware and verify:vmware. Merge the reviewed feature before release.
Deploy an existing KVM candidate	Run package:kvm when required. If infrastructure is missing, run optional infra:start. Click deploy:kvm, then verify:kvm; deploy:agent adds GitLab visibility.
Deploy a monolith candidate	Stop the other application target; click deploy:monolith and verify:monolith.
Install a full Debian package	Run package:deb first; select the target's DEB deployment mode. Deployment signs/uses the package rather than silently building one.
Publish a version	Use the successful protected-master pipeline; click release. Review tag, named release, registry assets and the release-branch integrations.

Storage maintenance

The CI project has a separate maintenance:retention pipeline, scheduled at 03:30 UTC by ci/retention.json. It keeps the two newest candidates per existing branch plus candidates protected by active jobs, successful release/deploy receipts or container references. Application pipelines do not contain a cleanup stage.

```
python3 scripts/retention.py  # read-only preview
```

Run the MyScoutee candidate retention schedule manually for immediate cleanup, or start a CI web pipeline with LAB_MAINTENANCE=retention. Release packages, Terraform modules/state, installer inputs, VM disks, credentials and persistent data are outside its deletion scope. Removing registry tags does not itself guarantee immediate layer-space reclamation.

MSC-129 qualification

Template monolith: pipeline 563, 14 checks passed. KVM: pipeline 567, 10 public checks passed, agent retry 2241 connected. VMware: pipeline 571, 11 passed / 0 failed / 3 HTTP checks skipped with the custom HTTPS tunnel. The operator accepted all three sequentially. Release and DEB exercise evidence is retained in verification.md; trial publications were withdrawn.

No Netcup deployment or execution of the newly introduced GitHub Actions conversion is claimed. That conversion remains under config/ci/github-actions.yml, outside active .github/workflows. Existing workflows retain their prior behavior.

Regenerate this PDF

```
python3 guides/publication/manuals/build-guide.py
```

Requires Python, ReportLab and DejaVu fonts. The logo and editable generator are committed in this directory. No frontend repository or running application is needed.